

Sri Lanka Institute of Information Technology



ASSIGNMENT

IT3051 - Fundamentals of Data Mining

Technical Report

Group: Mine4Data (DS. Y3S2.01.01)

Airbnb Listing Price Prediction

Student Names and Registration Numbers:

- **IT23404328**

Introduction and problem definition

Dataset

- Dataset Name: Airbnb Price Dataset Original Source: Compiled from Airbnb listing data across six major US cities (New York City, Los Angeles, San Francisco, Washington D.C., Chicago, and Boston).
- Published on Kaggle by Rupinder Singh Rana.
- Platform: Kaggle

Dataset URL/ Download source and citation

URL: <https://www.kaggle.com/datasets/rupindersinghrana/airbnb-price-dataset>

Citation: Rana, R.S. (n.d.). Airbnb Price Dataset. Kaggle. Retrieved September 2026, from <https://www.kaggle.com/datasets/rupindersinghrana/airbnb-price-dataset>

Licence: Publicly available on Kaggle for academic and research use

Dataset Description and problem

The Airbnb Price Dataset contains approximately 74,111 short-term rental listings across six major cities in the United States: New York City, Los Angeles, San Francisco, Washington D.C., Chicago, and Boston. Each record represents an individual Airbnb listing and includes property characteristics, host information, location data, review metrics, and booking policies.

The dataset was originally compiled from Airbnb's platform data and has been curated for price prediction tasks. The target variable ($\log_price$) is the natural logarithm of the nightly listing price in USD, a standard transformation applied to handle the heavy right-skew typical of rental price distributions.

Real-World Problem: New Airbnb hosts often struggle to set a competitive nightly price for their listings. Pricing too high leads to low occupancy; pricing too low leaves revenue on the table. A data-driven pricing recommendation tool would analyse comparable listings based on property attributes, location, and amenities to suggest an optimal nightly rate — helping hosts maximise occupancy and revenue while remaining competitive in their market.

This project builds a supervised regression system that suggests a nightly price for an Airbnb listing from its property, location and booking details.

Task type. Supervised regression on a continuous target. Success is measured by RMSE on the log scale (primary), supported by R^2 , MAE in dollars and the share of listings priced within $\pm 25\%$ of the real price.

Assigned scenario and user/stakeholder requirements

The scenario is a data-driven pricing recommendation tool: a host enters property details and receives a suggested nightly price based on comparable listings. The tool recommends; it does not set prices or value property.

Stakeholder	Decision Supposed	What they need from the system
New Airbnb hosts (primary user)	What nightly price to set for a new listing	A dollar price from details they know before any guest books, with no reviews or history required
Existing hosts	Whether to adjust price after upgrades or market changes	A quick re-estimate when bedrooms, amenities or policies change
Airbnb pricing teams	Which factors drive price across cities	Interpretable drivers (room type, capacity, location, amenities) and per-city error levels
Real-estate investors	Short-term rental income potential of a property	A typical nightly rate for a property profile in a given city

Functional Requirements

1. Accept listing inputs through a form: city, property type, room type, neighborhood, bedrooms, beds, bathrooms, accommodation, amenities, cancellation policy, cleaning fee and instant bookable.
2. Apply exactly the same preprocessing used in training, predict `log_price`, and return the price as "Suggested nightly price: \$XXX".
3. Use only features a host can supply for a brand-new listing (the deployable "listing" feature set), so no reviews or host history are needed.
4. Show a typical price with a range, and warn that very cheap and luxury listings are less reliable.

Non-Functional Requirements

- Accuracy: clearly beat a mean-price baseline (CV RMSE 0.719) and price most listings within $\pm 25\%$.
- Speed: return a prediction in well under a second, so the model must be fast at inference.
- Reproducibility: fixed train/test split and random seed (42) shared by every notebook and team member.
- No leakage: preprocessing fitted on training data only; test set used once, for the final model.
- Ethics: framed as a market-reflective recommendation, not a property valuation; no personal data stored.

Dataset identification, source, citation, and validation

The project uses the public Airbnb Price Dataset from Kaggle: 74,111 listings and 29 columns covering six US cities, with log_price as the target.

Item	Detail
Dataset name	Airbnb Price Dataset
Publisher / platform	Rupinder Singh Rana, Kaggle
URL	kaggle.com/datasets/rupindersinghrana/airbnb-price-dataset
Coverage	New York City, Los Angeles, San Francisco, Washington D.C., Chicago, Boston
Size	74,111 rows $\times$ 29 columns (single CSV)

Target	$\log_price = \ln(\text{nightly price in USD})$, continuous
Licence	Public on Kaggle for academic and research use

Validation of the dataset

- Source credibility: a widely used public Kaggle dataset compiled from Airbnb platform data; it contains no host names, guest details or contact information (no PII).
- Schema check: all 29 expected columns present with the types listed in the proposal; id, name, description and thumbnail_url were identified as non-predictive.
- Target check: log_price behaves as a log of price (e.g. \$150 $\rightarrow$ 5.01, \$75 $\rightarrow$ 4.32); $\exp(\log_price)$ returns plausible nightly prices from \$10 to about \$2,000.
- Row integrity: after cleaning, 74,109 listings remained and were split 80/20 into 59,287 training and 14,822 test rows; a saved split_ids.csv file confirms every notebook uses exactly the same split.
- Representativeness check: the five cross-validation folds have near-identical target mean (4.77–4.79), standard deviation (0.71–0.73), NYC share (43–44%) and entire-home share (55–56%).
- Known limitations: the 2017 snapshot does not reflect current prices, and review and host-history columns are not available for a brand-new listing (handled in Section 6).

Data understanding and EDA

EDA showed that price is driven first by room type, then location, then amenities and size, and that the raw price is heavily skewed, which justifies modelling log_price.

Target distribution. Raw nightly prices cluster between \$50 and \$300 with a long luxury tail up to about \$2,000. On the log scale the distribution is close to normal (training mean 4.78, standard deviation 0.72, i.e. a typical price of about \$119). Price deciles in the training set run from \$10–50 at the bottom to \$300–1,999 at the top.

Composition of the training set (59,287 listings)

Group	Listings	Share
NYC	25,940	43.8%
LA	17,882	30.2%
SF	5,134	8.7%
DC	4,551	7.7%
Chicago	2,972	5.0%
Boston	2,808	4.7%
Entire home/apt	33,112	55.9%
Private room	24,428	41.2%
Shared room	1,747	2.9%

NYC and LA hold 74% of listings, so location effects are learned most reliably there. Shared rooms are rare (under 3%), which later makes them the hardest segment to price.

Key EDA findings

1. **Room type** separates price levels most strongly: entire homes cost far more than private rooms, which cost more than shared rooms.
2. **Location** matters at several levels: city, neighbourhood, and distance to the city centre. Prices peak about 1–2 km from the centre and fall with distance.
3. **Size variables** (accommodates, bedrooms, beds, bathrooms) rise with price but are strongly correlated with each other, which signals multicollinearity for linear models.
4. **Amenities** are stored as a text set (e.g. {TV, Wifi, Kitchen}); specific amenities carry more signal than the raw count.
5. **Missing values** appear in review_scores_rating, host_response_rate, first_review/last_review (listings never reviewed), bathrooms, bedrooms, beds and zipcode.
6. **Outliers** exist in accommodates (16+), bathrooms (8+) and beds; they are mostly genuine large properties, not errors.
7. **Leakage risk**: number_of_reviews and review_scores_rating only exist after a listing has been active, so they cannot be used for a brand-new listing.

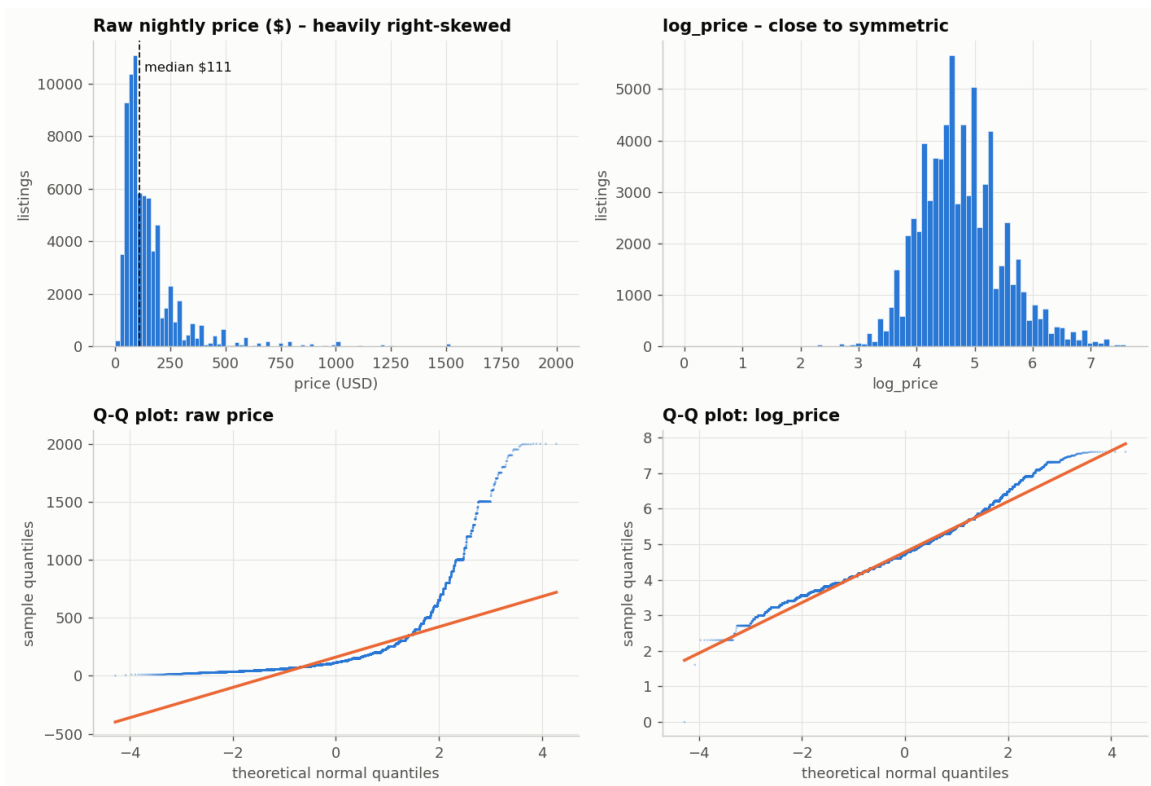


Figure: raw nightly price (right-skewed) versus log_price (close to normal)

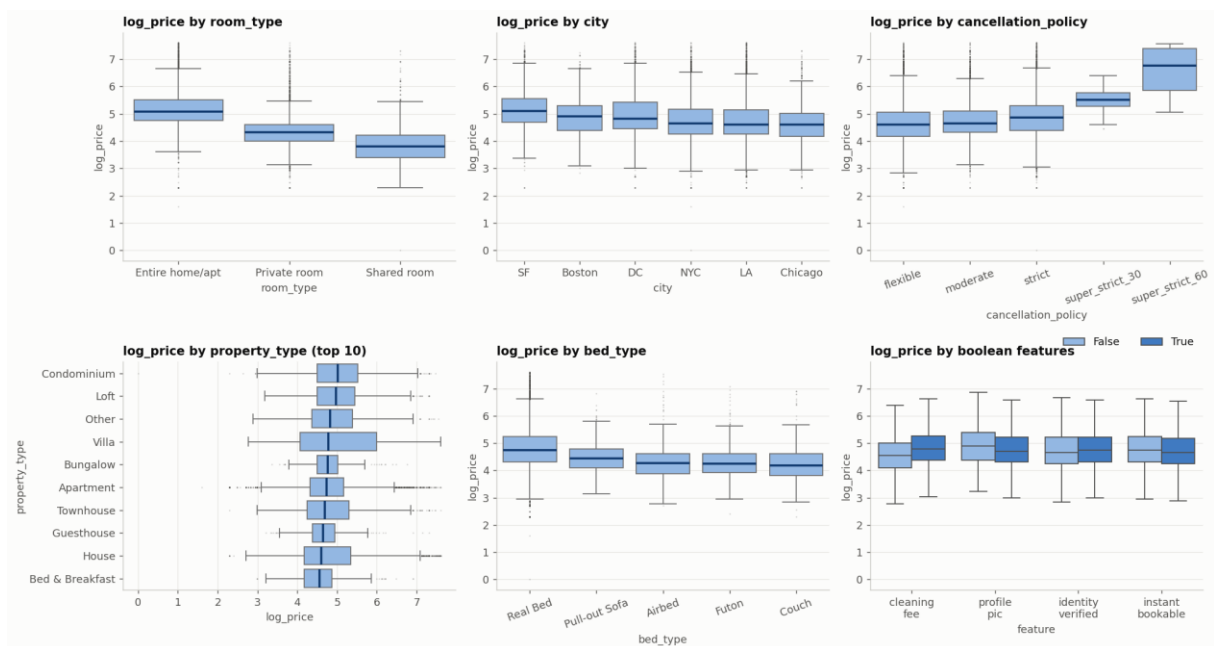


Figure: log_price by room type, city and other categorical features

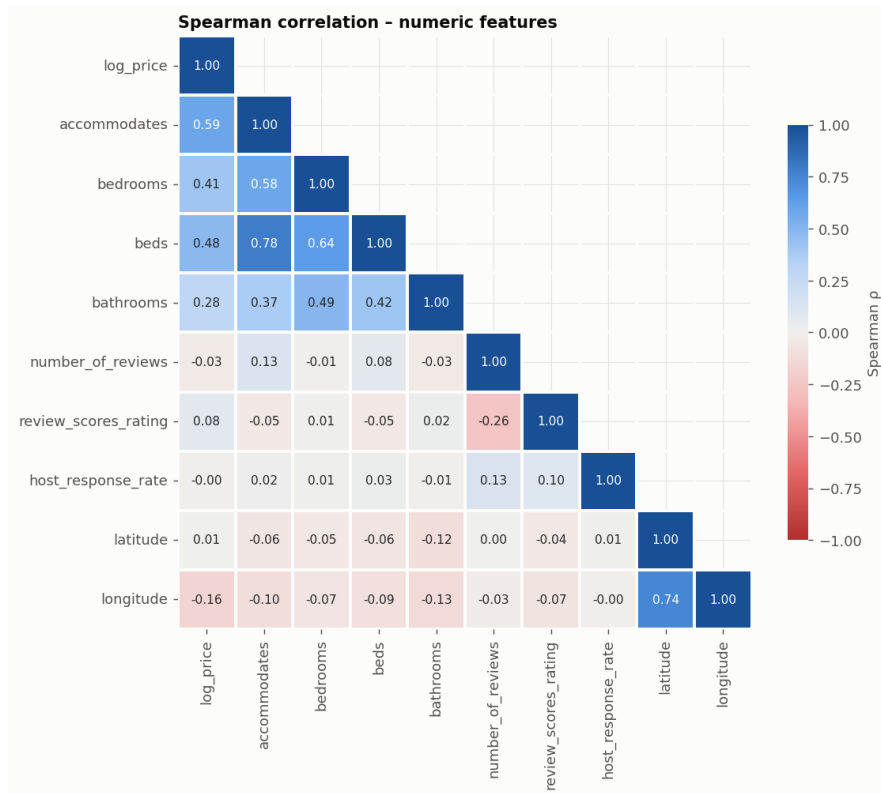


Figure: correlation of numeric features (size variables are strongly inter-correlated)

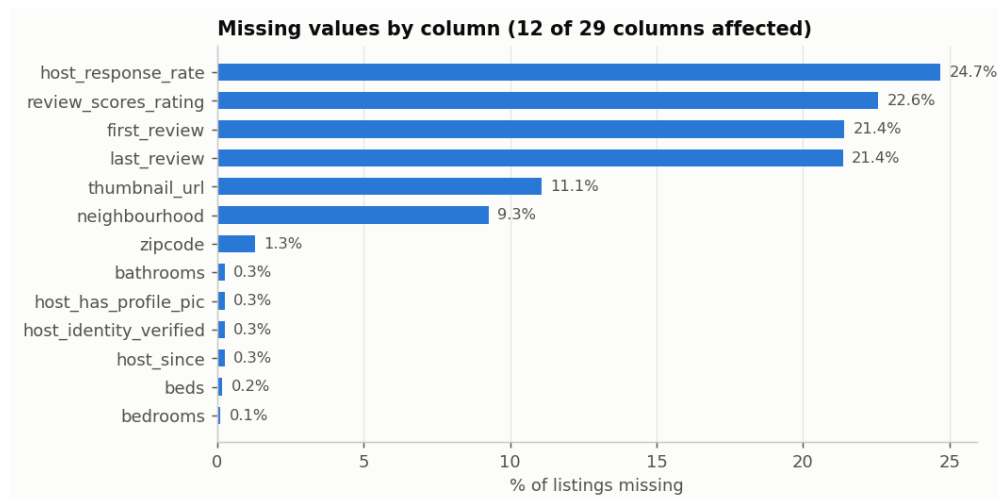


Figure: share of missing values per column

Listing locations coloured by log_price (★ = city centre used for distance)

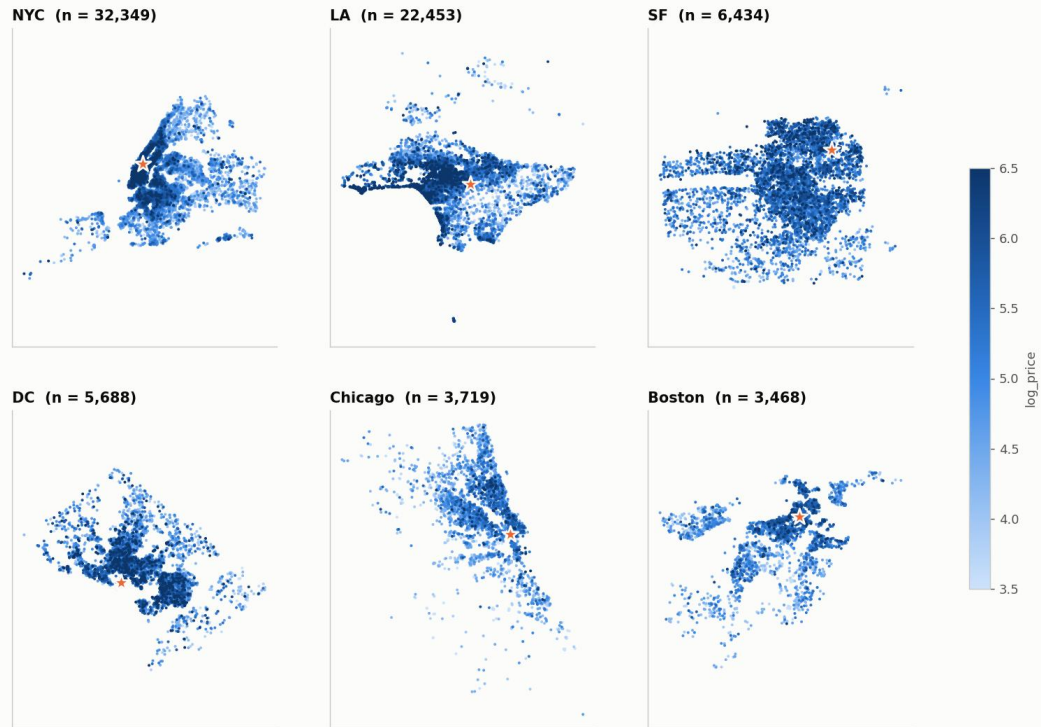


Figure: listings coloured by price in each city

Data cleaning and preprocessing

Cleaning reduced the data from 74,111 to 74,109 listings, and all learned preprocessing (imputation, encoding, scaling) runs inside an sklearn Pipeline so it is fitted on training folds only.

Cleaning steps (src/preprocessing.py, clean_and_engineer)

Issue	Columns	Treatment
Non-predictive columns	<code>id</code> , <code>name</code> , <code>description</code> , <code>thumbnail_url</code>	Dropped
Percentage stored as text	<code>host_response_rate</code>	Converted from "95%" to numeric 0.95
Booleans stored as "t"/"f"	<code>instant_bookable</code> , <code>host_has_profile_pic</code> , <code>host_identity_verified</code> , <code>cleaning_fee</code>	Converted to 0/1
Missing size values	<code>bathrooms</code> , <code>bedrooms</code> , <code>beds</code>	Median imputation (fitted on training data)
Missing reviews	<code>review_scores_rating</code> , <code>first_review</code> , <code>last_review</code>	Imputed, plus a has-reviews indicator (only in the "full" feature set)
Missing host response	<code>host_response_rate</code>	Imputed ("full" feature set only)
Ordered category	<code>cancellation_policy</code>	Ordinal code (<code>cancellation_ordinal</code> : flexible < moderate < strict)
Amenity text set	<code>amenities</code>	Parsed into binary flags (see Section 6)
Invalid rows	-	2 listings removed (74,111 → 74,109)

Train/test split. An 80/20 split stratified on price quintiles (`random_state = 42`) gave 59,287 training and 14,822 test listings, so both sets have the same price distribution. Listing IDs per split are saved to `data/processed/split_ids.csv`. The test set stayed locked until the final model was chosen.

Two preprocessing pipelines

Pipeline	Used by	Steps	Output columns
Linear	Linear Regression, Ridge, KNN	Impute → winsorise outliers (0.5–99.5%) → one-hot encode (first level dropped) → target-encode neighbourhood → standard-scale → correlation filter	87
Tree	Random Forest, XGBoost	Impute → one-hot encode (all levels) → target-encode neighbourhood → correlation filter; no scaling or winsorising	90

Linear models and KNN need scaling and outlier capping because penalties and distances depend on magnitude. Trees only compare values with thresholds, so scaling is unnecessary and outliers do little harm.

Leakage prevention. Preprocessing is the first step of each Pipeline, so in every cross-validation fold the medians, target encodings and scalers are learned on the four training folds only. The high-cardinality neighbourhood column is target-encoded inside the fold for the same reason.

Feature engineering and feature selection

Engineered features lower the final model's CV RMSE from 0.4002 to 0.3834; the amenity flags carry most of that gain, and keeping all 90 features beats any top-k subset.

Engineered features

Feature	How it is built	Why (evidence, training data only)
dist_centre_km	Haversine distance from the listing to its city centre; missing coordinates are first filled from the neighbourhood	Raw latitude has almost no linear signal ($\rho \approx 0.005$); distance does ($\rho = -0.27$); prices peak 1–2 km from the centre
guests_per_bathroom, guests_per_bedroom, beds_per_guest	Capacity ratios (a studio counts as one room)	Describe the shape of a listing; guests_per_bathroom ($\rho = 0.47$) is stronger than raw bathrooms (0.28)
is_studio	bedrooms = 0	Stops studios being read as “0 bedrooms = missing”
amenity_count and 53 amenity flags (amen_*)	Amenity text parsed, synonyms merged; one flag per amenity present in at least 2% of training listings	Which amenities a listing has matters more than how many (removing the flags costs +0.013 RMSE)
cancellation_ordinal	flexible = 0 < moderate = 1 < strict = 2 (super-strict merged into strict)	Ordered category; $\rho = 0.15$ with log_price
is_real_bed	bed_type = Real Bed	Simple quality signal from a 5-level category
neighbourhood (target-encoded)	Cross-fitted target encoding with smoothing, inside the pipeline	619 levels, too many for one-hot; main location signal when no coordinates are given
Host / review durations (full set only)	Years since host joined, first and last review	Not available for a brand-new listing, so used only in the “full” comparison set

Algorithms implemented and their rationale

Five algorithms from four families were compared, plus a mean baseline. Each family makes a different assumption about how price depends on the features, so comparing families shows what kind of structure the data has.

Algorithms compared

Model	Family	Why it suits this problem	Expected weakness	Preprocessing
Mean baseline	Reference (not counted)	Predicts the training mean for every listing; any useful model must clearly beat it.	Ignores all features	–
Linear Regression	Linear	Simplest, fully interpretable model. EDA showed strong, roughly straight-line effects on the log scale (capacity, room type).	No interactions; unstable with correlated size features	Linear pipeline: winsorised, scaled, first dummy dropped
Ridge	Linear (L2-regularised)	The L2 penalty stabilises the coefficients of correlated size features (accommodates, beds, bedrooms: $r \approx 0.7$ – 0.8) and gives a setting to tune.	Still additive and linear	Linear pipeline

Model	Family	Why it suits this problem	Expected weakness	Preprocessing
K-Nearest Neighbours	Instance-based	Mirrors how hosts price in practice: look at similar listings nearby. Assumes no functional form.	Every feature counts equally in the distance; slow to predict	Linear pipeline (scaling is essential for distances)
Random Forest	Bagging ensemble	Many independent deep trees, averaged. Captures non-linear effects and interactions (the effect of size depends on room type and city); robust to outliers.	Large model; averaging limits extreme predictions	Tree pipeline: unscaled, all dummies
XGBoost	Boosting ensemble	Trees built one after another, each correcting the previous errors, with built-in regularisation. A strong reference on tabular data.	Many hyperparameters	Tree pipeline

Why these five. Linear Regression is the plain reference, and Ridge shows whether regularisation fixes its problems. Random Forest and XGBoost are the two main ways of combining trees: averaging independent trees versus correcting errors step by step. Lasso and HistGradientBoosting were tried in an early run but left out of the final comparison, because they repeat families already covered (linear and boosting) and did not answer a new question.

Fair comparison set-up. All models used library defaults in the first round, so the algorithms, not the tuning effort, were compared. One documented exception: Random Forest used `max_features = 1/3` (Breiman's standard choice for regression; the default is about three times slower). Every model was trained on the same deployable “listing” feature set.

Validation strategy. 5-fold shuffled cross-validation on the 59,287 training listings (`random_state = 42`), with identical folds for every model and preprocessing refitted inside each fold. A time-based split was not needed: the data is a single 2017 snapshot and the task is pricing now, not forecasting.

Metrics

Metric	Role
RMSE (log) – primary	The quantity models <u>minimise</u> ; 0.1 log units $\approx$ 10% price error, and <u>squaring penalises large misses</u>
MAE (log)	Checks that RMSE is not driven by a few huge errors
R^2	Share of price variance explained
MAE and median AE (USD)	Average dollar error per night, which hosts understand
MAPE (%) and share within $\pm 25\%$	Business view: how often the suggestion is close enough to use

Model evaluation and comparison

At default settings the two tree ensembles tie for best (CV RMSE 0.393, R^2 0.701), clearly ahead of the linear models (0.417), KNN (0.447) and the mean baseline (0.719).

5-fold CV results, default settings, listing features (sorted by RMSE)

Model	RMSE (log)	RMSE std	R^2	MAE (\$)	Median AE (\$)	MAPE (%)	Within $\pm 25\%$ (%)	Fit time per fold (s)
XGBoost	0.3929	0.0067	0.701	49.9	22.7	29.6	57.0	8.4
Random Forest	0.3931	0.0077	0.701	50.0	22.3	29.1	57.3	32.3
Linear Regression	0.4168	0.0069	0.664	53.4	24.6	31.6	53.6	4.5
Ridge	0.4172	0.0070	0.663	53.5	24.6	31.6	53.5	7.2
KNN	0.4472	0.0079	0.613	56.4	26.9	34.9	50.6	2.6
Mean baseline	0.7191	0.0090	0.000	88.6	50.7	62.8	28.8	2.4

Is the ranking reliable? Because all models share the same folds, a paired fold-by-fold comparison was made. XGBoost beats the linear models by 0.024 and KNN by 0.054 on all 5 folds, with a spread of only 0.002, so these gaps are systematic. Against Random Forest the average difference is 0.0002 and XGBoost wins only 2 of the 5 folds, so the two ensembles are tied at defaults. No t-test was used: folds share training data, so it would overstate confidence.

Over- and under-fitting (train R^2 vs CV R^2)

Model	Train R^2	CV R^2	Gap	Diagnosis
Random Forest	0.939	0.701	0.238	Strong over-fitting
KNN	0.745	0.613	0.132	Mild over-fitting
XGBoost	0.790	0.701	0.088	Mild over-fitting
Linear Regression / Ridge	0.669	0.663–0.664	0.005	Under-fitting (high bias)

Why the ensembles win. Learning curves show Ridge's train and validation errors meet at about 0.41–0.42 even with 4.7k rows, so more data barely helps (bias-limited). XGBoost's validation error keeps falling from 0.435 to 0.393 as data grows from 4.7k to 47k rows (variance-limited). Price has non-linear effects and interactions, and 59k listings is enough to learn them.

Where errors come from

- **Price range:** all models over-predict the cheapest decile (about +0.33 to +0.42 log units) and under-predict the most expensive (about −0.45 to −0.54), i.e. regression toward the mean. XGBoost is closest to zero at both ends.
- **City:** NYC (27.2% MAPE) and LA (28.2%) are easiest; DC (43.0%) and SF (33.9%) are hardest.
- **Room type:** shared rooms are hardest (37.0% MAPE, only 1,747 listings). XGBoost beats Linear Regression in every segment.

Feature-set experiment. With the “full” set (adding reviews and host history), R^2 rises by 0.015 (Linear Regression) to 0.023 (XGBoost), but the ranking does not change. All nine Stage 6 runs are recorded as experiments S6-01 to S6-09 in results/stage6_experiment_log.csv.

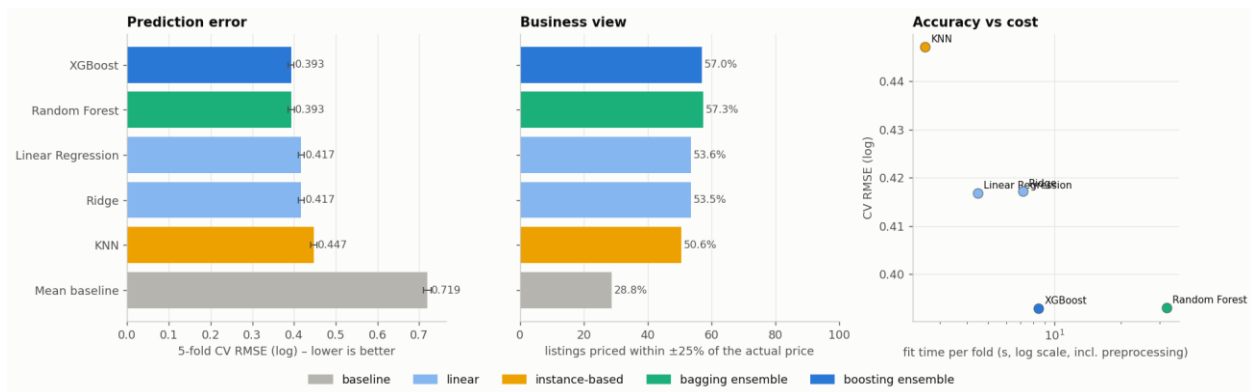


Figure: 5-fold CV comparison of the five models and the mean baseline

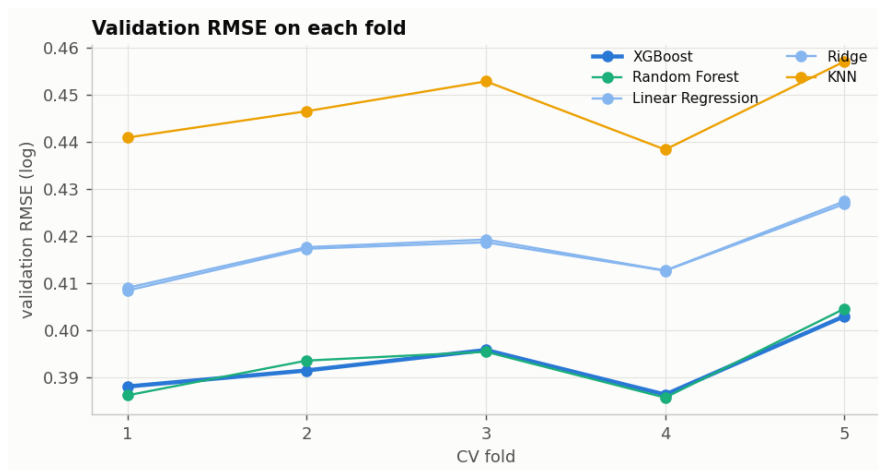


Figure: validation RMSE on each fold (same folds for every model)

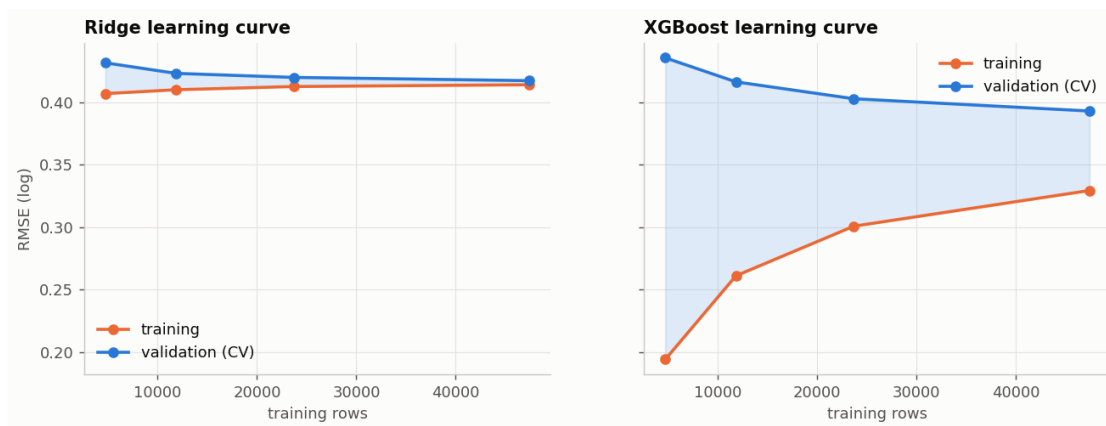


Figure: learning curves – Ridge is bias-limited, XGBoost keeps improving with more data

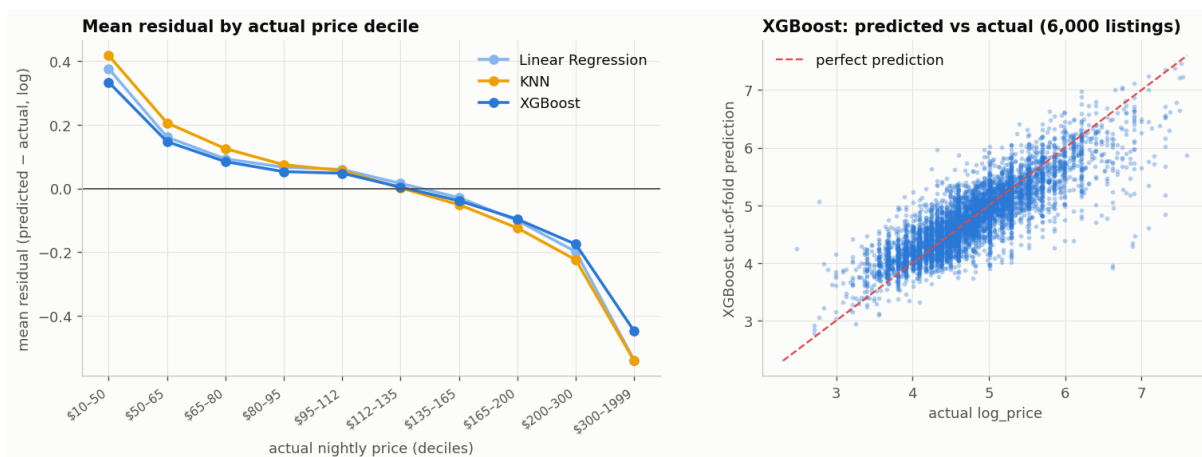


Figure: mean residual by actual price decile (regression toward the mean)

Hyperparameter tuning and optimization

Tuning made XGBoost the clear leader (CV RMSE 0.3929 $\rightarrow$ 0.3834, R^2 0.716) and separated the two ensembles that were tied at defaults; the family order did not change.

Search design. Grid search was used where there are one or two parameters with few sensible values; randomised search was used for the ensembles, where a full grid over 6–8 interacting parameters would need thousands of fits (Bergstra and Bengio, 2012). Every search:

- minimised 5-fold CV RMSE (log) on the training set only, using the same folds as Stage 6;
- searched the whole Pipeline, so preprocessing was refitted inside every fold;
- cached each fold's fitted preprocessor with joblib.Memory to save time without leaking validation data;
- used `random_state = 42` and saved every candidate to `results/stage7_search_<model>.csv`.

Model	Strategy	Search space	Best settings
Linear Regression	None	No hyperparameters	—
Ridge	Grid, 11 values	alpha 0.01–1,000 (log-spaced)	alpha = 0.01
KNN	Grid, 8 candidates	$k \in \{5, 10, 20, 40\} \times \text{weights} \in \{\text{uniform, distance}\}$	$k = 20$, distance-weighted
Random Forest	Random, 6 candidates	$n_estimators \in \{200, 300\}$; $max_features \in \{0.2, 1/3, 0.5\}$; $min_samples_leaf \in \{1, 2, 4, 8\}$; $max_depth \in \{\text{None}, 25\}$	200 trees, $max_features = 0.2$, $min_samples_leaf = 1$, $max_depth = 25$

Model	Strategy	Search space	Best settings
XGBoost	Random, 20 candidates	n_estimators 300–1,500; learning_rate 0.01–0.15 (log); max_depth 4–10; min_child_weight 1–29; subsample 0.6–1.0; colsample_bytree 0.4–1.0; reg_lambda 0.1–20 (log); reg_alpha 0.001–1 (log)	1,257 trees, learning_rate 0.022, max_depth 10, min_child_weight 15, subsample 0.84, colsample_bytree 0.48, reg_lambda 1.53, reg_alpha 0.004

Log-uniform distributions were used for learning rate and regularisation strengths because they act multiplicatively.

Tuned vs baseline (5-fold CV RMSE, log)

Model	Baseline RMSE	Tuned RMSE	Improvement	Tuned R ²	Candidates	Search time (s)
XGBoost	0.3929	0.3834	+2.4%	0.716	20	2,112
Random Forest	0.3931	0.3922	+0.2%	0.703	6	581
Linear Regression	0.4168	0.4168	0.0%	0.664	–	0
Ridge	0.4172	0.4168	+0.1%	0.664	11	142
KNN	0.4472	0.4283	+4.2%	0.645	8	307

What tuning revealed

- **Linear model:** Ridge error is flat (0.4168–0.4174) from alpha 0.01 to 100 and rises only for very strong penalties, confirming the linear models are bias-limited, not over-fitting. The chosen alpha = 0.01 is practically plain Linear Regression.
- **KNN:** a U-shaped validation curve (best at k = 20) shows the bias-variance trade-off; distance weighting wins at every k. Even tuned, KNN (0.428) trails XGBoost (0.383).
- **Random Forest:** all six candidates land within 0.3922–0.3945, so defaults were already near-optimal.
- **XGBoost:** the best candidates use small learning rates (about 0.02–0.05) with many trees and deep but regularised trees (min_child_weight 15, about half the features per tree); learning rates of 0.1 or more give the worst results (about 0.396–0.400).

Other optimisation experiments. Starting from tuned XGBoost, one choice was changed at a time (an ablation study) with the same folds and every learned step refitted per fold. Changes within ± 0.002 are treated as noise.

Feature engineering, feature selection and ensembling experiments (5-fold CV)

Experiment	Question	CV RMSE	Δ vs tuned XGBoost	Verdict
S7-01 reference: tuned, listing features	Reference	0.3834	–	reference
S7-02 without ratio features	Are the capacity ratios useful?	0.3828	-0.0006	no meaningful change
S7-03 without dist_centre_km	Is distance to the centre useful?	0.3846	+0.0011	no meaningful change

Experiment	Question	CV RMSE	Δ vs tuned XGBoost	Verdict
S7-04 without neighbourhood target encoding	Is the encoded neighbourhood useful?	0.3831	-0.0003	no meaningful change
S7-05 without amenity flags (keep count)	Are the 53 amenity flags useful?	0.3963	+0.0129	worse → keep as is
S7-06 without all engineered features	Is feature engineering useful overall?	0.4002	+0.0168	worse → keep as is
S7-07 full feature set (+ host & history)	How much do post-listing features add?	0.3632	-0.0203	better, but not usable for a new listing
S7-08 amenity vocabulary $\geq 1\%$ of listings	More amenity flags?	0.3831	-0.0003	no meaningful change
S7-09 amenity vocabulary $\geq 5\%$ of listings	Fewer amenity flags?	0.3830	-0.0004	no meaningful change
S7-10 feature selection: top 20 of 90	Does removing weak features help?	0.4045	+0.0211	worse → keep as is
S7-11 feature selection: top 40 of 90	Does removing weak features help?	0.3877	+0.0043	worse → keep as is
S7-12 average of XGBoost and Random Forest	Does averaging models help?	0.3842	+0.0007	no meaningful change

- Feature engineering matters overall (+0.017 without it), and the 53 amenity flags carry most of that gain: which amenities a listing has matters more than how many.
- Removing “weak” features hurts (top 40: +0.004, top 20: +0.021), so all 90 features are kept; XGBoost already ignores noise through column subsampling and regularisation.
- Averaging tuned XGBoost with Random Forest is slightly worse (0.3842, +0.0007): the two models make very similar errors, so there is little to cancel and the weaker forest pulls XGBoost down.
- The full feature set is better (−0.020) but needs reviews and host history that a brand-new listing does not have, so it is reported only as an upper bound.

Caveat. The tuning budgets were small (6 candidates for Random Forest, 20 for XGBoost), and a best-of-N CV score is slightly optimistic. The held-out test result in Section 10 shows no sign of this.

Final model selection and justification

The final model is tuned XGBoost on the listing features. On the untouched test set it scores RMSE 0.3808, R^2 0.711 and MAE \$47.81, and prices 59.1% of listings within $\pm 25\%$.

Selection rule (fixed before the test set was used). Choose the single tuned model with the lowest CV RMSE on the deployable features. An ensemble replaces it only if it lowers CV RMSE by more than 0.005, about the fold-to-fold noise. Averaging XGBoost with Random Forest made CV RMSE 0.0007 worse, so XGBoost alone was kept.

Selection criteria

Criterion	Evidence	Verdict
Accuracy	Lowest CV RMSE 0.3834 (R^2 0.716); Random Forest second at 0.3922	Best
Stability	Fold-to-fold standard deviation about 0.006, similar to the other models	Good
Ensembling	Averaging with Random Forest is 0.0007 worse (rule: must gain more than 0.005)	Single model is enough
Speed	About 70 ms to price one listing, including all preprocessing	Good
Size	24.5 MB in memory, 10.1 MB compressed on disk; Random Forest would be 816 MB	Good
Deployability	Uses only the 15 form fields; the same pipeline object as in cross-validation	Good
Interpretability	Less transparent than a linear model; explained with permutation importance and partial dependence	Acceptable

Why not the alternatives. Linear models are interpretable but 0.033 worse in RMSE (about \$4.8 more error per night), and the gap is bias that tuning cannot remove. KNN is the weakest tuned model (0.428) and needs the whole training set at prediction time. Random Forest is less accurate (0.392), about twice as slow per prediction and 33 times larger. An ensemble adds complexity for no measurable gain.

Final test-set evaluation (used once)

Metric	CV estimate (train)	Test - <u>XGBoost</u>	Test - mean baseline
RMSE (log)	0.3834	0.3808	0.7088
R^2	0.716	0.711	0.000
MAE (USD)	48.62	47.81	86.80
Median AE (USD)	-	21.53	50.38
MAPE (%)	-	28.2	61.8
Within $\pm 25\%$ (%)	58.4	59.1	29.1

Test and CV results agree closely, so the CV procedure was honest and tuning did not over-fit the folds. The model cuts average dollar error by about 45% compared with the baseline. Scored after the decision was locked, every tuned model's test RMSE is within 0.0034 of its CV value and the ranking is unchanged. Residuals are centred near zero (mean -0.006 , std 0.381).

Error by price band (test set)

Actual price band	Listings	Median actual	Median predicted	Median abs. % error
10–65 USD	3,035	50	61	23.7
65–95 USD	2,894	80	84	16.4
95–135 USD	3,032	111	115	18.0
135–200 USD	3,036	165	159	17.7
200–1,975 USD	2,825	300	245	27.0

The middle 60% of listings have 16–18% median error; budget and luxury listings are less reliable, so the app shows a price range and a warning for them.

What the model relies on (permutation importance, test RMSE increase when shuffled)

Rank	Input	RMSE increase
1	Room type	0.127
2	<u>Neighbourhood</u>	0.041
3	Amenity flags (all)	0.036
4	Bedrooms	0.029
5	Distance to <u>centre</u>	0.025
6	Longitude	0.024
7	Accommodates	0.020
8	Latitude	0.018
9	Bathrooms	0.014
10	Property type	0.007

Price is set first by the kind of space, then by location, then amenities, then size. Partial dependence agrees with intuition: the average predicted price rises from about \$136 for 1 guest to \$188 for 10 guests, and peaks around 1–2 km from the city centre (about \$167) before falling to about \$126 beyond 30 km.

Saved for deployment. The full pipeline (preprocessing + tuned XGBoost) is saved as `models/final_model.joblib` (10.1 MB) with `final_model_metadata.json` (form fields, hyperparameters, metrics, library versions). A reload test priced a 2-bed Williamsburg NYC apartment at \$183, a private room in the same flat at \$80 and the same apartment in Venice, LA at \$199, and matched the in-memory model exactly.

System architecture and implementation

The system has three layers: a React web page used by the host, a FastAPI prediction service, and the model files saved at the end of Stage 7. For the demonstration a single uvicorn process on port 8010 serves both the built web page and the API, so the whole system starts with one command.

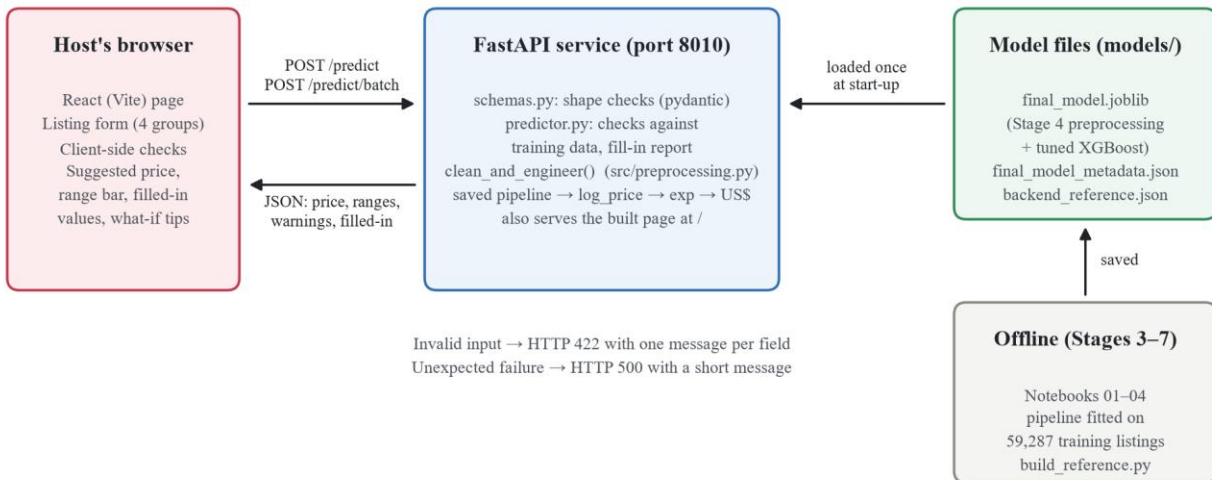


Figure: system architecture and request flow

Components

Layer	Files	Responsibility
Frontend	frontend/src/App.jsx, components/*.jsx, listing.js, whatif.js, api.js	Input form, client-side checks, calls to the API, result display and what-if tips
API layer	backend/app.py, backend/schemas.py	Endpoints, request shape validation (pydantic), uniform error responses, serving the built page
Prediction layer	backend/predictor.py	Loads the model once, checks values against the training data, applies the same preprocessing, converts log_price to US\$, builds the result
Rule-based preprocessing	src/preprocessing.py (clean_and_engineer)	The same cleaning and feature-engineering function used in Notebooks 02–04
Model artefacts	models/final_model.joblib, final_model_metadata.json, backend_reference.json	The fitted Stage 7 pipeline, its metadata and the lookup data for validation
Offline tools	Notebooks 01–04, backend/build_reference.py	Train and save the model; rebuild the reference file after retraining

How one request is handled

- **Load once at start-up.** `final_model.joblib` is the whole Stage 7 sklearn Pipeline (Stage 4 learned preprocessing + tuned XGBoost), so nothing is refitted in the service. If any model file is missing the service refuses to start instead of giving wrong prices.
- **Validate.** `pydantic` checks types, required fields and unknown fields; `predictor.py` checks values against `backend_reference.json` (valid categories, neighbourhoods per city, city coordinate bounds and value ranges, all taken from the training split).
- **Same preprocessing as training.** The validated input becomes a raw listing row, goes through `clean_and_engineer()` and then through the saved pipeline, exactly as the test set did in Notebook 04.
- **Predict and convert.** The model returns `log_price`, which is converted to US\$ with `exp()`.
- **Return.** A JSON result with the price, two price ranges, the optional fields that were filled in, warnings and the model's test metrics.

Key design decisions

One saved pipeline, not re-implemented preprocessing. Imputation, rare-level grouping, target encoding and the amenity encoder are stored inside `final_model.joblib`, so the service cannot drift from what was evaluated. A test confirms that the API returns the same prices as the Notebook 04 pipeline.

Price ranges from unseen data. The likely range (middle 50%) and wide range (middle 80%) are the quantiles of actual/predicted price on the held-out test set: $0.81\text{--}1.21\times$ and $0.66\text{--}1.55\times$ the suggested price. They are measured on data the model never saw, so they are honest.

Transparency about missing inputs. Only city, room type, property type and number of guests are required. Every other field the host leaves out is filled in by the trained pipeline (training medians or most common values) and listed back to the user as “filled in”.

Backend and frontend development

Backend (FastAPI)

The backend is a REST API written with FastAPI and served by uvicorn. It loads the final model once and exposes five endpoints; interactive documentation is generated automatically at `/docs`.

Endpoint	Purpose
GET /health	Service is up and the model is loaded
GET /model-info	Final model, hyperparameters and test metrics
GET /options	Valid values for every input (cities, neighbourhoods per city, amenities, numeric ranges); used to build the frontend form
POST /predict	Suggested price for one listing
POST /predict/batch	Prices for up to 100 listings in one call (used for the what-if tips)

Input validation. Required fields are city, room_type, property_type and accommodates. The service rejects unknown fields, wrong types, unknown categories (with “did you mean” suggestions), a neighbourhood that belongs to another city, coordinates outside the chosen city, values outside the training range and bathrooms that are not whole or half numbers. All problems are reported in one response, so the user can fix everything at once. Unrecognised amenity names are ignored with a warning, so a typo cannot inflate the amenity count.

Error handling. Invalid input returns HTTP 422 with one message per field; an unexpected failure returns HTTP 500 with a short message and is logged. Example: a request with city “Paris”, property type “Apartmnt” and 40 guests returns:

Field	Message returned by the API
city	'Paris' is not recognised; choose one of: Boston, Chicago, DC, LA, NYC, SF
property_type	'Apartmnt' is not recognised. Did you mean: Apartment?
accommodates	must be between 1 and 16 (got 40)

Response. A successful prediction returns predicted_price (US\$ per night), likely_range and wide_range, log_price, filled_in (each missing optional field, the value used and why), warnings (for example “No neighbourhood or coordinates given: the location is estimated from the city as a whole”), the cleaned input and the model's test R^2 and median error.

Frontend (React)

The frontend is a single React page built with Vite and styled for a non-technical host. It talks only to the API, so all prediction logic stays in one place.

- **Clear inputs.** Short numbered instructions at the top and four groups of fields: location, the place, booking rules and amenities. Required fields are marked *; everything else can be left as “Not sure”. All dropdown values come from GET /options, so the user can only choose values the model knows.
- **Validation.** Required fields, number ranges, whole or half bathrooms and coordinates (given together and inside the chosen city) are checked before sending. The backend checks everything again and its messages are shown next to the same fields.
- **Clear results.** The suggested price, “most similar listings charge between \$X and \$Y” with a bar showing the 50% and 80% ranges, notes, the values that were filled in, and a plain sentence on reliability: half of test-set suggestions were within \$22 of the real price.
- **What-if tips.** The page asks the model about one realistic change at a time in a single batch call (adding a common amenity, turning on Instant Book, hosting one more guest) and shows the five changes that raise the price most, stating that these are associations, not guaranteed increases.
- **Usability details.** If the form is edited after a result, an “Update price” banner appears so an old price is never shown as current; the layout adapts to phone screens and the result scrolls into view.

Test group	What is checked	Tests	Result
Invalid or missing inputs	Each required field missing (4); invalid values such as unknown city, misspelt property type or neighbourhood, out-of-range numbers, 1.3 bathrooms (12); wrong types (4); neighbourhood in another city; coordinates outside the city; only one coordinate; unknown field; all problems reported at once; bad JSON or empty body; batch errors show the item index	27	Pass
Frontend logic (Vitest)	Form validation, form-to-API payload conversion, server errors mapped to the right fields, what-if variants and ranking	13	Pass
Total		49	49 passed

Scenario tests (sensible behaviour)

Example listings sent to POST /predict show that the price responds to inputs in the direction a host would expect:

Listing	Suggested price	Likely range (50%)
NYC, Williamsburg, entire apartment, 4 guests, 2 bedrooms, 1 bathroom	\$181	\$147 – \$219
Same flat offered as a private room for 2 guests	\$80	\$65 – \$97
Same flat offered as a shared room for 1 guest	\$65	\$52 – \$78
Same apartment in Chicago (Logan Square)	\$124	\$100 – \$149
Same apartment, 6 guests, 3 bedrooms, 2 bathrooms	\$311	\$252 – \$376
Only the required fields: LA, private room in a house, 2 guests	\$96 (10 fields filled in, 2 warnings)	\$78 – \$116

Performance and end-to-end checks

- **Speed:** over 50 requests on a laptop, POST /predict took a median of 80 ms (95th percentile 90 ms), including validation and preprocessing; a batch of 40 what-if variants took about 100 ms. This meets the requirement of a prediction in well under a second.
- **End to end:** with the built page served by the API, the form loads its options, shows field-level errors, returns a price with ranges and tips, shows the “Update price” banner after edits, and works on a phone-width screen.
- **Model quality in the deployed system** equals the final test-set result: RMSE 0.381 (log), R^2 0.711, median error \$21.53 per night and 59.1% of listings within $\pm 25\%$ of the real price, against 29.1% for the mean baseline.

Limitations and possible future improvements

Limitations

- **Old and narrow data.** The dataset is a 2017 snapshot of six US cities. Prices have changed since then, and the model should not be used for other cities or countries without retraining.
- **Extreme prices are pulled toward the middle.** Budget (under \$65) and luxury (over \$200) listings have 24–27% median error, against 16–18% for the middle 60%. Shared rooms and listings in DC and SF are the weakest segments.
- **Information the data does not record.** Photos, decor, view, exact building and host strategy affect price but are not in the data, which limits R^2 to about 0.71. Reviews and host history would raise it to about 0.745 but do not exist for a brand-new listing.
- **No time dimension.** The data has no booking dates or occupancy, so the tool suggests one typical nightly price and cannot account for seasons, events or weekdays, or optimise revenue.
- **Associations, not causes.** The what-if tips show how prices of similar listings differ, not the guaranteed effect of a change.
- **Prototype deployment.** The system runs locally without authentication, rate limiting or monitoring, and the saved model depends on the library versions it was trained with.

Possible future improvements

- Retrain on recent public Airbnb data (for example Inside Airbnb) and add calendar, seasonality and occupancy data, so the tool can suggest prices per date and estimate revenue.
- Use quantile regression or conformal prediction to give price ranges that adapt to each segment instead of one global ratio.
- Add text and image features (listing description, photos) to capture quality differences the current features miss.
- Collect more data or build segment-specific models for shared rooms, DC and SF; widen the tuning budget with Bayesian optimisation.
- Explain each individual prediction with SHAP values so hosts see why their price is what it is.
- Deploy to the cloud in a container with authentication, logging, drift monitoring and scheduled retraining.

Individual and group contributions

Fill in the names and registration numbers of the group members before submission.

Stage / deliverable	Main work	Member(s)
Problem definition and dataset proposal	Scenario, stakeholders, requirements, dataset validation	[name, IT number]
EDA	Notebook 01: structure, data quality, distributions, relationships, leakage risks	[name, IT number]
Preprocessing and feature engineering	Notebook 02, src/preprocessing.py: cleaning, split, pipelines, engineered features	[name, IT number]
Model development	Notebook 03, src/modelling.py: five algorithms, cross-validation, comparison	[name, IT number]
Optimisation and final model	Notebook 04: tuning, ablation, final model, test evaluation	[name, IT number]
Backend	backend/, tests/test_backend.py: prediction API and tests	[name, IT number]
Frontend	frontend/: React page, validation, results, what-if tips	[name, IT number]
Report and presentation	Technical report, slides and demonstration	[name, IT number]

References

- Bergstra, J. and Bengio, Y. (2012) ‘Random search for hyper-parameter optimization’, *Journal of Machine Learning Research*, 13, pp. 281–305.
- Breiman, L. (2001) ‘Random forests’, *Machine Learning*, 45(1), pp. 5–32.
- Chen, T. and Guestrin, C. (2016) ‘XGBoost: a scalable tree boosting system’, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- Hoerl, A.E. and Kennard, R.W. (1970) ‘Ridge regression: biased estimation for nonorthogonal problems’, *Technometrics*, 12(1), pp. 55–67.
- Micci-Barreca, D. (2001) ‘A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems’, *ACM SIGKDD Explorations Newsletter*, 3(1), pp. 27–32.
- Pedregosa, F. et al. (2011) ‘Scikit-learn: machine learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Rana, R.S. (n.d.) Airbnb Price Dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/rupindersinghrana/airbnb-price-dataset> (Accessed: September 2026).
- Ramírez, S. (n.d.) FastAPI documentation. Available at: <https://fastapi.tiangolo.com> (Accessed: October 2026).
- Meta Platforms (n.d.) React documentation. Available at: <https://react.dev> (Accessed: October 2026).